



TMLC
ACADEMY

LLM Tracing



Large Language Models (LLMs) are powerful but often function as black boxes. As developers, we need visibility into how prompts are processed, where latencies occur, and how different components interact. This is where **LLM tracing** comes in. It allows us to monitor, debug, and optimize our AI applications, ensuring efficiency and reliability.

In this blog, we'll explore **LLM tracing**, why it's essential, and how tools like **Langsmith**, **Opik**, and **Langfuse** can help track and analyze model interactions.

What is LLM Tracing?

LLM tracing refers to the process of **tracking and visualizing** the different steps involved in handling a request through an LLM-powered pipeline. This includes:

- **Input Processing:** How the prompt is structured and passed to the model.
- **Model Invocation:** Time taken by the model to generate responses.
- **Intermediary Steps:** Calls to external APIs, vector database queries, or function executions.
- **Response Handling:** Formatting, ranking, or post-processing is applied before sending back the response.

Why Do We Need LLM Tracing?

- **Debugging Issues:** Identify bottlenecks, failed requests, and incorrect outputs.
- **Optimizing Performance:** Reduce latency and resource consumption.
- **Understanding Model Behavior:** Gain insights into how prompts affect responses.
- **Logging and Analytics:** Track user interactions and improve model responses.

LangSmith

When LangChain / LangGraph was first created, its primary goal was to simplify prototyping for LLM applications. However, moving from prototype to production introduces new challenges, requiring a more structured and reliable approach.

This is where LangSmith comes in.

A simple way to think about it:

 LangChain / LangGraph = Prototyping
 LangSmith = Production

But what makes production different from prototyping?

One major challenge is reliability. While it's relatively easy to build an LLM application that works well in a controlled environment, achieving consistent and dependable performance at scale remains a significant hurdle.

To address this, LangSmith focuses on five key areas:

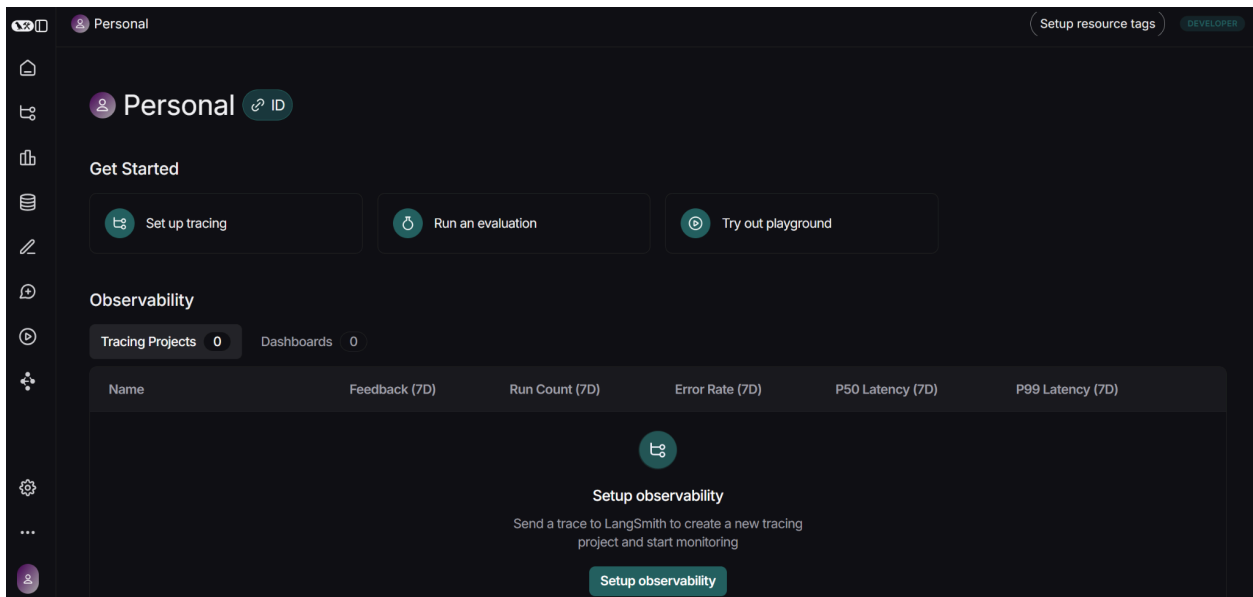
- ✓ Debugging
- ✓ Testing
- ✓ Evaluating
- ✓ Monitoring
- ✓ Usage Metrics

A major advantage of LangSmith is its intuitive UI, which lowers the barrier to entry—especially for those without a software engineering background—making it easier to manage and optimize LLM applications in production.

Let's code something now,

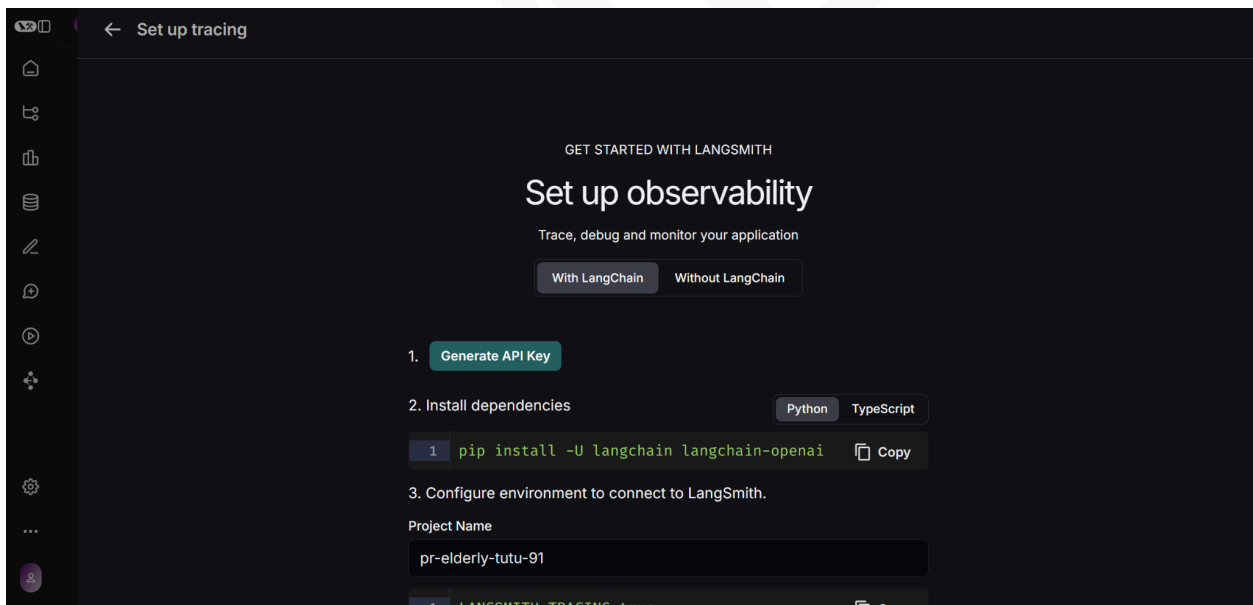
1. Login to [LangSmith](#)

An UI like this shall be visible



2. LangSmith API Key

To Get LangSmith API Key Click **Setup Tracing**



Generate the API key and give a project Name

3. Setup LangSmith with existing LangChain, LangGraph RAG or Agents.

We will take an example of Wikipedia Agent here, but before that install langsmith

pip install langsmith

```
import os
import dotenv
import pandas as pd
from langchain.agents import initialize_agent, AgentType, load_tools
from langchain_community.chat_models import ChatOpenAI
from langchain_core.tools import Tool
from langchain.chat_models import ChatOpenAI
from langchain import hub
from langchain.memory import ConversationBufferMemory

dotenv.load_dotenv()
os.environ['OPENAI_API_KEY'] = os.getenv('OPENAI_API_KEY')
os.environ['LANGSMITH_API_KEY'] = os.getenv('LANGSMITH_API_KEY')
os.environ['LANGSMITH_PROJECT'] = os.getenv('LANGSMITH_PROJECT')
os.environ['LANGSMITH_ENDPOINT'] = os.getenv('LANGSMITH_ENDPOINT')
os.environ['LANGSMITH_TRACING'] = os.getenv('LANGSMITH_TRACING')

llm = ChatOpenAI(temperature=0.1, model_name="gpt-4o-mini")
tools = load_tools(["wikipedia"], llm=llm)

memory = ConversationBufferMemory(memory_key="chat_history")

conversational_agent = initialize_agent(
    agent='conversational-react-description',
    tools=tools,
    llm=llm,
    verbose=True,
    max_iterations=3,
    memory=memory,
)

response = conversational_agent.run("Who won the 2022 Fifa world cup?")
print(response)
```

4. The .env file

```
OPENAI_API_KEY=<Add your key>
LANGSMITH_TRACING=true
```

```
LANGSMITH_ENDPOINT=https://api.smith.langchain.com
LANGSMITH_API_KEY=<Add your key>
LANGSMITH_PROJECT=<Add your project name>
```

5. Results

Run the code be it in colab or a .py file

The screenshot shows the LangSmith interface for a project named 'genai-guided-projects'. The 'Runs' tab is active, displaying a table with one run. The run is named 'AgentExecutor' and has an input of 'Who won the ...' and an output of 'Argentina ...'. The start time is '2/21/2025, 2:...' and the latency is '7.36s'. The tokens used are '1,847'. On the right side, there are filter shortcuts for 'Input Key', 'Input Key Value', 'Name', 'Run Type', 'Status', and 'Tag'.

Name	Input	Output	Error	Start Time	Latency	Dataset	Annotation Queue	Tokens
AgentExecutor	Who won the ...	Argentina ...		2/21/2025, 2:...	7.36s			1,847

The results can be viewed at Tracing panel in the left side with the project name one has defined. It will show the input, output, latency, tokens used and probable cost.

Click on All runs to get a detailed view of all the components involved

The screenshot shows the LangSmith interface for the same project 'genai-guided-projects'. The 'Runs' tab is active, and the 'All Runs' filter is selected. The table displays multiple runs for different components: 'ChatOpenAI', 'LLMChain', 'wikipedia', and 'AgentExecutor'. Each run shows the input, output, error, start time, latency, and dataset. On the right side, there are filter shortcuts for 'Input Key', 'Input Key Value', and 'Input Value'.

Name	Input	Output	Error	Start Time	Latency	Dataset
ChatOpenAI	human: Assistant is a large language ...	ai: Do I need to use a...		2/21/2025, 2:5...	1.48s	
LLMChain	Who won the 2022 Fifa world cup?	Do I need to use a to...		2/21/2025, 2:5...	1.49s	
wikipedia	2022 FIFA World Cup ...	Page: 2022 FIFA Wor...		2/21/2025, 2:5...	4.39s	
ChatOpenAI	human: Assistant is a large language ...	ai: ... Thought: Do I n...		2/21/2025, 2:5...	1.39s	
LLMChain	Who won the 2022 Fifa world cup?	... Thought: Do I nee...		2/21/2025, 2:5...	1.39s	
AgentExecut...	Who won the 2022 Fifa world cup?	Argentina won the 2...		2/21/2025, 2:5...	7.36s	

Dataset and Evaluation

LangSmith also supports creating dataset and testing on the dataset, but it might be the best as compared to libraries created specifically for evaluation like DeepEval, Ragas, Truelens etc..

Check pre-defined tutorials [here](#).

🌟 Conclusion

As LLM applications scale, **tracing becomes essential** to maintain performance, reliability, and user satisfaction. Tools like **Langsmith** provide the necessary observability to track, debug, and optimize AI workflows.

🔍 If you're building an AI-powered product, consider integrating LLM tracing today to gain insights and improve performance!